

name:m.naga shalini
 reg no:192524183
 course:csa0406-operating systems

1.A deadlock is a situation in which two or more processes are unable to continue execution because each process is waiting for a resource that is held by another process.
 In the given pseudo-code, Process P1 first locks Resource R1 and waits for 100 ms before trying to lock Resource R2.
 At the same time, Process P2 first locks Resource R2 and also waits for 100 ms before trying to lock Resource R1.
 The execution takes place as follows:
 Process P1 successfully locks R1.
 Process P2 successfully locks R2.
 Both processes wait for 100 ms.
 After the wait, P1 attempts to lock R2, but it cannot because R2 is already locked by P2.
 Similarly, P2 attempts to lock R1, but it cannot because R1 is already locked by P1.
 Now, P1 is waiting for R2 while holding R1, and P2 is waiting for R1 while holding R2.
 Since neither process releases its first resource until it obtains the second resource, both processes wait indefinitely. This creates a deadlock.
 The exact condition causing the deadlock is the Circular Wait condition. P1 waits for a resource held by P2, while P2 waits for a resource held by P1, forming a circular chain. The deadlock also satisfies the Hold and Wait condition because each process holds one resource while waiting for another.
 Since the resources cannot be shared (Mutual Exclusion) and cannot be forcibly taken away (No Preemption), all four necessary conditions for deadlock are satisfied.
 Optimization to eliminate the deadlock without changing the process logic:
 The best optimization is to enforce a fixed resource ordering.
 Both processes should acquire the resources in the same order, for example, always lock R1 first and then R2.
 With this approach, if one process has already locked R1, the other process must wait until R1 becomes available instead of locking R2 first.
 As a result, the Circular Wait condition is eliminated, preventing deadlock while keeping the overall program logic unchanged.
 Another possible optimization is to use try-lock with timeout or deadlock detection, where a process releases its resource and retries if it cannot obtain the second resource within a specified time. However, resource ordering is the simplest and most efficient solution.
 Conclusion:
 The deadlock occurs because P1 locks R1 and waits for R2, while P2 locks R2 and waits for R1, creating a circular wait.
 By ensuring that all processes acquire resources in the same order (R1 → R2), the circular wait condition is removed, and the deadlock is completely prevented without changing the intended functionality of the processes.

2.CPU utilization is the percentage of time the CPU is actively executing processes. In the given problem, the CPU utilization is only 30%, even though processes are available in the ready queue.
 The scheduler log shows repeated state transitions:
 Ready → Waiting → Ready → Waiting → Ready → ...
 This indicates that the processes are frequently entering the waiting state instead of continuously using the CPU.
 The main reason is that the processes are I/O-bound, meaning they spend more time waiting for I/O operations (such as disk access, keyboard input, or network communication) than performing CPU computations.
 Whenever a process requests an I/O operation, it moves from the Running state to the Waiting state.
 After the I/O operation is completed, it returns to the Ready state and waits again for CPU allocation.
 This repeated switching between the Ready and Waiting states causes the CPU to remain idle for a significant

amount of time, resulting in poor CPU utilization.

Another possible cause is frequent context switching. Since processes repeatedly enter the waiting state,

the operating system must continuously save and restore process states.

Excessive context switching increases

scheduling overhead, reducing the time available for actual execution.

Improper scheduling policies or a shortage

of CPU-intensive processes can also contribute to low CPU utilization.

To improve CPU utilization, the operating system should implement

multiprogramming. By increasing the number

of processes in memory, the CPU can execute another ready process

whenever one process is waiting for I/O.

This ensures that the CPU remains busy instead of staying idle. T

he scheduler can also use efficient scheduling algorithms such as

Round Robin or Multilevel Feedback Queue (MLFQ) to distribute CPU time

fairly and reduce unnecessary waiting.

In addition, techniques such as asynchronous I/O, buffering, caching,

and spooling can reduce I/O waiting time,

allowing processes to spend more time executing on the CPU.

Conclusion:

The frequent transition Ready → Waiting → Ready indicates that the

processes are I/O-bound, causing the CPU to remain

idle despite the presence of ready processes.

The best system-level optimization is to use multiprogramming,

along with efficient CPU scheduling and improved I/O management techniques.

These optimizations keep the CPU busy, reduce idle time,

increase throughput, and improve overall system performance.

3.A page fault occurs when a program tries to access a page that is not currently available in the physical memory (RAM).

The operating system must then load the required page from secondary storage (disk) into RAM.

Frequent page faults increase memory access time and significantly reduce the performance of the application.

In the given memory map:

Code segment: 5 MB

Heap: 150 MB

Stack: 1 MB per thread

Number of threads: 25

Physical RAM available: 2 GB

The total stack memory used by the threads is:

$25 \times 1 \text{ MB} = 25 \text{ MB}$

Therefore, the total memory used is:

Code = 5 MB

Heap = 150 MB

Stack = 25 MB

Total Memory Used = $5 + 150 + 25 = 180 \text{ MB}$

Although 180 MB is much smaller than the available 2 GB RAM,

the application still experiences frequent page faults.

This indicates that the performance issue is not caused by insufficient physical memory, but by poor memory access patterns.

In a multi-threaded application, several threads may access different parts of the heap simultaneously,

causing poor cache locality and unnecessary page replacements.

Large dynamic memory allocations and scattered heap usage can also increase page faults.

Frequent thread switching further reduces cache efficiency and increases memory management overhead.

Optimizations

1. Optimize Memory Usage

Reduce unnecessary heap allocations and use efficient memory management techniques such as memory pooling,

object reuse, and contiguous memory allocation.

This improves memory locality and reduces the number of page faults.

2. Reduce Thread Overhead

Instead of creating 25 separate threads, use a thread pool

or reduce the number of active threads if possible.

Fewer threads consume less stack memory, reduce context switching,

improve cache performance, and lower memory management overhead.

Conclusion

The performance degradation is caused by frequent page faults due to inefficient memory access and excessive multi-threading, rather than a lack of physical RAM. Optimizing heap memory usage and reducing thread overhead through techniques such as memory pooling and thread pools can significantly reduce page faults and improve overall system performance.

4.

Starvation is a condition in which a process waits indefinitely for CPU time because other higher-priority processes continuously receive the CPU. In the given multilevel queue scheduler, here are three queues:

Q1 (Highest Priority): Interactive Processes (Time Quantum = 5 ms)

Q2: System Processes (Time Quantum = 10 ms)

Q3: Batch Processes (FCFS)

Process P5 is placed in Q3, the lowest-priority queue. The scheduler always gives preference to processes in Q1 and Q2.

If new interactive or system processes keep arriving, they are executed first, preventing P5 from receiving CPU time.

As a result, P5 remains in the ready queue for an indefinite period, leading to starvation. The root cause of starvation is the strict priority scheduling used by the multilevel queue scheduler.

Since higher-priority queues are always served before lower-priority queues, the batch process in Q3 is continuously postponed.

Although P5 is ready to execute, it never gets a chance because higher-priority processes keep entering the system.

Optimizations

1. Aging Technique

Implement aging, where the priority of a process is gradually increased as its waiting time increases.

If P5 waits for a long time in Q3, its priority is increased, allowing it to move to a higher-priority queue and eventually receive CPU time.

Aging is the most effective method to prevent starvation.

2. Fair CPU Scheduling

Allocate a fixed percentage of CPU time to each queue or periodically schedule processes from lower-priority queues.

This ensures that even batch processes in Q3 receive CPU time, improving fairness and preventing indefinite postponement.

Conclusion

The starvation occurs because P5 belongs to the lowest-priority queue (Q3), while higher-priority queues continuously receive CPU time.

Implementing aging and fair CPU scheduling ensures that every process eventually gets CPU time, preventing starvation and improving overall system fairness.

5. A context switch occurs when the operating system saves the state of the currently running process and loads the state of another process.

Although context switching is necessary for multitasking, excessive context switching reduces system performance

because the CPU spends more time switching between processes than executing them.

In the given server:

20,000 context switches per second

Average CPU burst time = 2 ms

Very high I/O wait frequency

Preemptive scheduler Time quantum = 1 ms

The main reason for the high number of context switches is the very small time quantum.

Since the average CPU burst time is 2 ms,

but the scheduler allows a process to run for only 1 ms, almost every process is interrupted before completing its CPU burst.

This causes frequent preemption and increases the number of context switches.

Another reason is the high I/O wait frequency. Processes frequently move from the Running state to the Waiting state to perform I/O operations.

Once the I/O operation is complete, they return to the Ready state and must be scheduled again.

These repeated transitions create additional context switches and increase CPU overhead.

Optimizations

1. Increase the Time Quantum

Increase the time quantum from 1 ms to a larger value such as 4 ms or 5 ms.

A larger quantum allows processes to complete their CPU bursts with fewer interruptions, reducing unnecessary context switches and improving CPU efficiency.

2. Reduce I/O Waiting

Use techniques such as asynchronous I/O, buffering, caching, or thread pools to reduce the number of I/O operations and waiting time.

This decreases the frequency of process state changes and lowers the number of context switches.

Conclusion

The high context-switch overhead is caused by the very small time quantum and frequent I/O waits. Increasing the time quantum and optimizing I/O operations significantly reduce context switching, improve CPU utilization, and enhance the overall performance of the server.